

# Airbnb MapReduce Analysis: A Comprehensive Multi-Dataset Pipeline Implementation

## Abstract

*This report presents a comprehensive MapReduce analysis pipeline for processing Inside Airbnb datasets using Hadoop Streaming with Python implementations. The system processes three complementary datasets (listings, reviews, and calendar) through eight distinct MapReduce jobs to analyze pricing patterns, budget accommodation supply, guest activity levels, and occupancy metrics across San Francisco neighborhoods. The modular pipeline demonstrates scalable big data processing techniques while producing actionable insights for hospitality market analysis.*

## I. Dataset Description

This analysis utilizes the Inside Airbnb open dataset, which provides curated snapshots of Airbnb activity across major cities worldwide. The dataset is published as CSV/CSV.GZ files organized by city and date, offering comprehensive insights into short-term rental markets.

**Source:** Inside Airbnb (<https://insideairbnb.com/get-the-data/>)

**Project Scope:** Complete Airbnb ecosystem analysis covering three primary datasets

- **Listings:** Property details, pricing, and host information
- **Reviews:** Guest review records with dates and listing associations
- **Calendar:** Daily availability and pricing data for each property

## A. Dataset Components

**Listings Dataset** contains property identifiers, location data (**neighbourhood\_cleansed**), property characteristics (**room\_type**, **bedrooms**, **bathrooms**, **beds**), pricing information, booking requirements (**minimum\_nights**), and performance metrics (**number\_of\_reviews**, **availability metrics**).

**The Reviews Dataset** provides guest feedback records, including **listing\_id** associations, enabling analysis of guest satisfaction patterns and booking frequency over time.

**Calendar Dataset** offers daily availability snapshots showing booking status, pricing fluctuations, and seasonal patterns for each property, supporting occupancy rate calculations and demand analysis.

**Analysis Objectives:** The expanded pipeline computes average price by neighborhood and room type combinations, identifies budget-friendly listings, ranks neighborhoods by affordable supply availability, analyzes cross-dataset relationships through join operations, calculates neighborhood-level review volumes, and determines occupancy patterns from calendar data.

## II. Setup

The project employs Hadoop Streaming to execute Python-based MapReduce jobs on the distributed file system. The setup process revealed several important configuration considerations and potential pitfalls.

### A. Environment Configuration

The Hadoop Streaming JAR is configured in *airbnb\_sf\_map\_reduce/bin/env.sh* with the path *HSTREAM\_JAR=\${HADOOP\_HOME}/share/hadoop/tools/lib/hadoop-streaming-\*.jar*. The HDFS namespace defaults under */projects/DATA-228-Projects/hw1/airbnb\_sf\_map\_reduce* with organized subpaths for raw data, clean preprocessed data, and various analysis outputs (*avg\_by\_nb\_rt*, *budget\_supply*, *budget\_supply\_ranked*).

### B. Key Setup Gotchas and Solutions

**Java Version Compatibility:** Ensuring the Java version matches your Hadoop distribution is crucial, as version mismatches can cause cryptic startup failures that are difficult to diagnose.

**Streaming JAR Path Resolution:** The HSTREAM\_JAR path expansion must work correctly. It is essential to verify that the wildcard resolves to an actual file using commands like *ls \${HSTREAM\_JAR}*.

**Python Environment Consistency:** Confirming that python3 is available on all cluster nodes is vital, as inconsistent Python installations across nodes can cause unexpected job failures.

**HDFS Permissions:** Proper read/write permissions for HDFS directories are mandatory. Permission errors often manifest as vague "job failed" messages that do not clearly indicate the underlying access issue.

**Compressed Input Handling:** The system correctly processes .csv.gz files via TextInputFormat, but it is important to verify that decompression works properly in your specific environment.

**Memory Configuration:** For laptop-based pseudo-distributed setups, YARN container memory limits may need adjustment if jobs fail during startup due to insufficient resources.

### C. Build Pipeline

The project uses a comprehensive Makefile-driven approach supporting multiple dataset processing modes:

#### Basic Pipeline Setup:

- ***make init*** - Initialize HDFS directory structure
- ***make fetch*** - Download InsideAirbnb CSV.gz files and upload to HDFS
- ***make check RAW=<dataset>*** - Verify required HDFS inputs exist

#### Preprocessing Targets:

- ***make data\_prep RAW=listings*** - Clean listings data to normalized 11-column TSV
- ***make prep\_reviews*** - Clean reviews data for join operations
- ***make prep\_calendar*** - Clean calendar data for occupancy analysis

#### Core Analytics:

- ***make avg*** - Average price by neighborhood and room type
- ***make budget*** - Budget supply analysis ( $\leq \$150$ ,  $\leq 7$  nights)
- ***make rank*** - Rank neighborhoods by budget supply availability

#### Cross-Dataset Analytics:

- ***make reviews\_by\_nb*** - Join listings with reviews to calculate total reviews per neighborhood
- ***make occupancy*** - Join listings with calendar to determine booked/total days per neighborhood

#### Pipeline Execution Modes:

- ***make all RAW=listings*** - Complete listings pipeline
- ***make all RAW=reviews*** - Reviews pipeline with cross-dataset joins
- ***make all RAW=calendar*** - Calendar pipeline with occupancy analysis
- ***make all RAW=all*** - Full multi-dataset pipeline with all analytics

## III. Data Preprocessing

The preprocessing stage transforms raw Inside Airbnb CSV data across three datasets into clean, validated formats suitable for both individual analysis and cross-dataset join operations.

### A. Multi-Dataset Processing

**Listings Preprocessing (data\_prep):** Processes raw listings CSV files from the raw/listings/ directory, implementing robust data cleaning and validation for the primary property dataset.

**Reviews Preprocessing (prep\_reviews):** Cleans reviews data from raw/reviews/ directory, focusing on extracting listing associations and temporal patterns for join operations with listings data.

**Calendar Preprocessing (prep\_calendar):** Processes calendar data from raw/calendar/ directory, normalizing availability status and pricing information to support occupancy rate calculations.

## B. Data Cleaning Logic

The preprocessing mappers implement dataset-specific cleaning while maintaining consistency for join operations. For listings data, the mapper parses CSV rows while handling formatting issues, extracts, and normalizes key fields, including listing IDs, neighborhoods (converted to lowercase), and room types. Price parsing handles currency symbols, commas, and decimal formatting robustly.

For review data, the preprocessing focuses on extracting clean listing\_id associations, parsing review dates, and creating temporal groupings (yyyy-mm format) to support time-based analysis.

Calendar preprocessing normalizes availability flags (converting 't'/'f' to 1/0), handles price variations including blank entries, and ensures consistent listing\_id formatting for successful joins with listings data.

## C. Validation Rules

**Listings Validation:** All records must have valid ID, neighbourhood\_cleansed, and room\_type fields. Price values must fall within \$0 to \$5000 range to exclude outliers. Minimum nights must be non-negative integers, and neighborhood names are normalized to lowercase.

**Reviews Validation:** Requires valid listing\_id that can be matched with listings dataset, valid date formatting, and basic data completeness checks.

**Calendar Validation:** Ensures listing\_id consistency with listings data, validates date formatting, and handles availability status normalization for accurate occupancy calculations.

## D. Example Output Schemas

**Cleaned Listings (11-column TSV):** listing\_id | neighbourhood | room\_type | price | min\_nights | num\_reviews | baths | bedrooms | beds | avail\_30 | avail\_365

**Cleaned Reviews (3-column TSV):** listing\_id | date | yyyy-mm

**Cleaned Calendar (4-column TSV):** listing\_id | date | is\_available | price\_or\_blank

All preprocessing jobs use identity reducers since cleaning logic resides in mappers, with outputs written to respective `clean_*` directories for downstream analytics and join operations.

## IV. Example Walkthrough

To illustrate the complete data processing pipeline, this section traces a sample dataset through each MapReduce step, showing how raw data transforms into final analytical insights.

### A. Sample Input Data

Starting with raw CSV data from Inside Airbnb:

```
id,neighbourhood_cleansed,room_type,price,minimum_nights,number_of_reviews,bathrooms_text,bedrooms,beds,availability_30,availability_365
12345,"Mission District","Entire home/apt","$180.00",2,45,1.0,1,1,15,200
67890,"SOMA","Private room","$85.00",1,12,1.0,1,1,25,300
11111,"Mission District","Private room","$120.00",3,8,1.0,1,1,10,150
22222,"Castro","Entire home/apt","$220.00",2,23,1.5,2,2,20,180
```

### B. Data Preprocessing Pipeline

#### Step 1: Mapper Input Processing

The preprocessing mapper receives each CSV line and performs data cleaning:

Raw line: 12345,"Mission District","Entire home/apt","\$180.00",2,45,1.0,1,1,15,200

#### Mapper processing:

- Parse CSV fields handling quotes and commas
- Normalize neighborhood: "Mission District" → "mission district"
- Parse price: "\$180.00" → 180.0
- Validate: id exists, neighborhood valid, room\_type valid,  $0 < \text{price} < 5000$ ,  $\text{min\_nights} \geq 0$
- Extract additional metrics: reviews=45, baths=1.0, bedrooms=1, beds=1, avail30=15, avail365=200

**Mapper output (TSV):** 12345 mission district Entire home/apt 180.0 2 45 1.0 1 1 15 200

Complete preprocessing output for sample data:

```
12345 mission district Entire home/apt 180.0 2 45 1.0 1 1 15 200
67890 soma Private room 85.0 1 12 1.0 1 1 25 300
11111 mission district Private room 120.0 3 8 1.0 1 1 10 150
22222 castro Entire home/apt 220.0 2 23 1.5 2 2 20 180
```

## C. Job 1: Average Price Analysis Walkthrough

**Mapper Input:** Cleaned TSV data from preprocessing

**Mapper Logic:** Extract neighborhood, room\_type, price and emit composite keys

**Mapper processing for each line:**

- Line 1: mission district Entire home/apt 180.0 1
- Line 2: soma Private room 85.0 1
- Line 3: mission district Private room 120.0 1
- Line 4: castro Entire home/apt 220.0 1

**Reducer Input:** Hadoop sorts and groups by key (neighborhood + room\_type)

**Reducer Processing:**

- Group 1: "castro + Entire home/apt" → Average: 220.00, Output: castro Entire home/apt 220.00 1
- Group 2: "mission district + Entire home/apt" → Average: 180.00, Output: mission district Entire home/apt 180.00 1
- Group 3: "mission district + Private room" → Average: 120.00, Output: mission district Private room 120.00 1
- Group 4: "soma + Private room" → Average: 85.00, Output: soma Private room 85.00 1

## D. Multi-Dataset Extensions

Starting with raw calendar entries showing availability and pricing fluctuations, the preprocessing normalizes availability flags and handles price variations. The calendar rollup job calculates booked versus total days per listing, which then joins with listings data to produce neighborhood-level occupancy metrics.

Reviews Analysis Example:

Reviews preprocessing extracts listing associations and temporal patterns. The listings-reviews join operation matches property locations with review activity, producing neighborhood-level guest satisfaction and booking frequency insights.

## V. MapReduce Jobs

The analysis pipeline consists of eight distinct MapReduce jobs organized into three categories: core analytics on individual datasets, cross-dataset join operations, and aggregation jobs for neighborhood-level insights.

## A. Core Analytics Jobs

### Job 1: Average Price by Neighborhood and Room Type (avg\_by\_nb\_rt)

Calculates mean pricing for each neighborhood-room type combination from cleaned listings data. The mapper processes cleaned TSV data, extracting neighborhood, room type, and price information, then emits composite keys pairing neighborhoods with room types alongside price values and count indicators. The reducer aggregates price sums and counts for each combination, computing average prices and outputting results in the format: neighborhood, room\_type, avg\_price, listing\_count.

### Job 2: Budget Supply Analysis (budget\_supply)

Counts affordable listings per neighborhood using specific budget criteria: price  $\leq$  \$150 per night and minimum nights  $\leq$  7 days (suitable for short stays). The mapper applies these budget filters to each listing, emitting neighborhood names with count indicators for qualifying properties. The reducer aggregates counts per neighborhood, producing output with the format: neighborhood, budget\_listing\_count.

### Job 3: Ranking Budget Supply (budget\_supply\_ranked)

Sorts neighborhoods by budget listing availability in descending order. The mapper creates inverted numeric keys to achieve descending sort order, while the reducer drops the sort keys and outputs the final ranked results in the format: neighborhood, budget\_listing\_count (sorted by count, highest first).

## B. Cross-Dataset Join Operations

### Job 4: Listings-Reviews Join (join\_listings\_reviews)

Performs inner join between cleaned listings and cleaned reviews datasets on listing\_id. The mapper processes both datasets, emitting listing\_id as the key with tagged values indicating source dataset (listings or reviews). The reducer matches listings with their associated reviews, outputting neighborhood and review counts to intermediate storage.

### Job 5: Listings-Calendar Join (join\_listings\_calendar)

Joins cleaned listings with calendar rollup data to associate occupancy metrics with neighborhood information. Similar to the reviews join, this mapper emits listing\_id keys with tagged values, and the reducer matches listings with their calendar statistics, producing intermediate output for aggregation.

## C. Aggregation and Rollup Jobs

### **Job 6: Reviews Aggregation (agg\_nb\_counts)**

Aggregates intermediate review join results to produce total review counts per neighborhood. The mapper emits neighborhood keys with review counts, while the reducer sums all review activity by neighborhood.

### **Job 7: Calendar Rollup Per Listing (cal\_rollup\_per\_listing)**

Processes cleaned calendar data to calculate booked and total days for each individual listing. The mapper emits listing\_id keys with availability status indicators, and the reducer counts booked days versus total days per listing.

### **Job 8: Occupancy Aggregation (agg\_occupancy\_by\_nb)**

Aggregates the joined listings-calendar data to produce neighborhood-level occupancy statistics. The mapper processes intermediate occupancy data, emitting neighborhood keys with booked and total day counts. The reducer sums these metrics across all listings within each neighborhood.

## **VI. Results and Insights**

### **A. Expanded Dataset Analysis**

The enhanced pipeline processes three complementary datasets, producing comprehensive insights into San Francisco's short-term rental ecosystem through both individual dataset analysis and cross-dataset relationships.

### **B. Core Dataset Results**

#### **Cleaned Dataset Schemas:**

- **Listings:** 11-column standardized TSV enabling neighborhood-level pricing and supply analysis
- **Reviews:** 3-column format optimized for temporal analysis and listing association joins
- **Calendar:** 4-column format supporting occupancy rate calculations and availability pattern analysis

**Pricing Analysis Results:** The average price analysis reveals significant neighborhood-based pricing variation with clear segmentation by property type. Entire homes consistently command premiums over private and shared rooms, with geographic location serving as a primary pricing determinant.



## C. Cross-Dataset Insights

**Guest Activity Patterns:** The reviews-by-neighborhood analysis reveals correlations between review volume and neighborhood popularity, indicating guest satisfaction levels and booking frequency patterns across different areas of San Francisco.

**Occupancy Rate Analysis:** Calendar data integration provides occupancy insights showing seasonal demand variations and neighborhood-specific booking patterns. The booked-to-total day ratios reveal which neighborhoods maintain higher utilization rates.

**Market Segmentation Depth:** Cross-dataset analysis enables identification of neighborhoods that combine favorable pricing, high guest satisfaction (review volume), and strong occupancy rates, providing comprehensive market positioning insights.

## D. Budget Accommodation Supply

The budget supply analysis ( $\leq \$150$ ,  $\leq 7$  nights minimum) identified neighborhoods with significant concentrations of accessible short-term rental options. The ranking system reveals geographic clustering patterns where budget-friendly accommodations concentrate, supporting both traveler planning and host strategy development.

## E. Neighborhood Performance Metrics

The integrated analysis produces neighborhood scorecards combining pricing competitiveness, guest activity levels (review counts), and property utilization rates (occupancy metrics). This multi-dimensional view enables more nuanced market understanding than single-dataset analyses.

# VII. Conclusion

This comprehensive MapReduce analysis demonstrates an advanced, multi-dataset Hadoop Streaming workflow that progresses systematically through data ingestion, cleaning, individual dataset analytics, cross-dataset joins, and neighborhood-level aggregations. The robust preprocessing implementation and sophisticated join operations produced integrated analytical outputs suitable for business intelligence and market strategy applications.

The expanded pipeline effectively addresses complex questions about San Francisco's Airbnb ecosystem: pricing patterns by location and property type, budget accommodation availability, guest activity levels through review analysis, and occupancy patterns via calendar data integration. The modular architecture supports extensible analytics while maintaining computational efficiency across multiple datasets.

## A. Key Analytical Achievements

The analysis revealed that San Francisco's short-term rental market exhibits strong neighborhood-based differentiation across multiple dimensions: pricing (with entire homes commanding significant premiums), guest satisfaction patterns (through review volume analysis), and utilization rates (via occupancy data). The integration of three datasets provides unprecedented insight depth compared to single-dataset approaches.

Cross-dataset join operations successfully linked property characteristics with guest behavior patterns and booking performance metrics, enabling identification of neighborhoods that optimize across price competitiveness, guest satisfaction, and occupancy success. This multi-dimensional analysis supports both traveler decision-making and host strategy development.

## B. Technical Architecture Success

The project successfully implemented a scalable big data processing pipeline using Hadoop Streaming with Python, demonstrating practical application of distributed computing principles to real-world hospitality industry datasets. The pipeline processes multiple data sources (listings, reviews, calendar) through coordinated MapReduce jobs with sophisticated join and aggregation logic.

The modular job design pattern facilitates easy extension to additional analysis questions while maintaining data processing reliability. The robust data cleaning processes handle real-world CSV complexity, including formatting variations and data quality issues across multiple source datasets.

## C. Pipeline Scalability and Reproducibility

The comprehensive Makefile automation supports flexible execution modes (single datasets, specific analytics, or complete multi-dataset pipelines) while maintaining reproducible results. The standardized HDFS namespace organization and clear job dependencies enable reliable execution across different Hadoop environments.

The workflow design accommodates datasets from other cities or time periods without modification, supporting longitudinal analysis and comparative market studies. The extensible architecture allows additional InsideAirbnb datasets (such as neighborhood boundaries or host information) to be integrated using the established join patterns.

## References

[1] InsideAirbnb, "Get the Data," 2024. [Online]. Available: <https://insideairbnb.com/get-the-data/>

**[2]** Apache Software Foundation, "Hadoop Streaming," Apache Hadoop Documentation, 2024.  
[Online].

Available: <https://hadoop.apache.org/docs/current/hadoop-streaming/HadoopStreaming.html>